Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015118	Course Name:	Artificial Intelligence & Soft Computing Lab

Name of Student:	Rushikesh Subhash Khemnar
Roll No. :	81
Experiment No.:	4
Title of the Experiment:	Implement Adversarial Search Using the Min-max Algorithm
Date of Performance:	10/8/26
Date of Submission:	17/8/26

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Signature :

Date :

Aim: To implement adversarial search using the Min-Max algorithm.

Objectives:

After completing this experiment, the student will be able to:

1. To understand the concept of adversarial search in Artificial Intelligence.
2. To study the working principle of the Min-Max algorithm.
3. To implement the Min-Max algorithm for two-player game scenarios.
4. To determine the optimal move using game tree evaluation.
5. To understand decision-making in competitive environments.

Tools Required:

- Hardware: o Desktop/Laptop
- Software:
  - o Python 3.x o Visual Studio Code / PyCharm / Jupyter Notebook o Windows/Linux Operating System

Theory:

Adversarial search is a decision-making technique used in Artificial Intelligence for solving competitive problems involving two or more players with opposing objectives. Unlike conventional search problems, the outcome of an adversarial search depends on the actions of both players. These techniques are commonly applied in strategy-based games such as TicTac-Toe, Chess, Checkers and Connect Four.

The Min-Max algorithm is a widely used adversarial search algorithm that determines the optimal move by constructing a game tree and evaluating all possible future states. It assumes that one player (MAX) attempts to maximize the utility value, while the opposing player (MIN) attempts to minimize it. By recursively evaluating the game tree, the algorithm selects the move that provides the best possible outcome for the MAX player, assuming that the MIN player always makes optimal decisions.

Procedure:

1. Define the game environment and represent it as a game tree.
2. Identify the MAX player and the MIN player.
3. Assign utility values to the terminal nodes.
4. Implement the Min-Max algorithm using recursion.
5. Execute the algorithm to evaluate all possible game states.
6. Identify the optimal move for the MAX player.

7. Record the output and verify the result.

Description of the Experiment:

This experiment demonstrates the implementation of the Min-Max algorithm for solving twoplayer game problems. A game tree is constructed to represent the possible moves of both players, and the algorithm recursively evaluates each game state to determine the optimal move.

The experiment provides an understanding of adversarial search, game tree traversal and decision-making strategies used in Artificial Intelligence for competitive environments.

Detailed Description:

The Min-Max algorithm is used to determine the optimal decision in a competitive environment where two players make alternate moves. The game is represented as a tree in which each node corresponds to a possible game state and the leaf nodes contain utility values representing the outcome of the game. The algorithm recursively evaluates the game tree by alternating between maximizing and minimizing operations.

The MAX player selects the move that yields the highest utility value, whereas the MIN player attempts to minimize the utility of the MAX player by selecting the lowest-valued move. By evaluating all possible game states, the Min-Max algorithm identifies the optimal strategy for the MAX player assuming that both players make rational decisions throughout the game.

Code:-

```
def minimax(node, maximizing_player):

    if isinstance(node, int):
        return node


    if maximizing_player:
        best_value = float('-inf')

        for child in node:
```

```
            value = minimax(child, False)
            best_value = max(best_value, value)

        return best_value

else:
        best_value = float('inf')

        for child in node:
            value = minimax(child, True)
            best_value = min(best_value, value)

        return                  best_value
game_tree = [
    [[3, 5], [2, 9]],
    [[0, 7], [4, 6]]
]


move_values = []

for i, move in enumerate(game_tree):      value =
minimax(move, False)      move_values.append(value)
print(f"Move {chr(65 + i)} has Min-Max value: {value}")


best_move_index          =          move_values.index(max(move_values))
best_move = chr(65 + best_move_index)

print("\nOptimal move for MAX:", best_move) print("Optimal
utility value:", move_values[best_move_index])
```

Output:-

```
C:\Users\student\Desktop\python>python min.py
Move A has Min-Max value: 5
Move B has Min-Max value: 6

Optimal move for MAX: B
Optimal utility value: 6
```

Conclusion:

The Min-Max algorithm was successfully implemented to perform adversarial search in a twoplayer game environment. The experiment demonstrated how optimal decisions are obtained through game tree evaluation and recursive search.

Questions to be Answered by Students:

1. Which player attempts to maximize the utility value in the Min-Max algorithm? The MAX player attempts to maximize the utility value.

2. What is the role of the terminal nodes in the game tree?
   Terminal nodes represent the final game states and contain utility values that indicate the outcome of the game.
3. Which optimal move was selected by the Min-Max algorithm for the given problem?
   The optimal move selected for the MAX player was Move B, with a utility value of 4.

4. State two observations obtained from the implementation of the Min-Max algorithm.

   - The Min-Max algorithm evaluates the game tree recursively by alternating between MAX and MIN operations.

   - The algorithm selected Move B because it provided the highest guaranteed utility value of 4 for the MAX player.